5.2 Low-Level Arrays

To accurately describe the way in which Python represents the sequence types, we must first discuss aspects of the low-level computer architecture. The primary memory of a computer is composed of bits of information, and those bits are typically grouped into larger units that depend upon the precise system architecture. Such a typical unit is a ***byte***, which is equivalent to 8 bits.

A computer system will have a huge number of bytes of memory, and to keep track of what information is stored in what byte, the computer uses an abstraction known as a ***memory address***. In effect, each byte of memory is associated with a unique number that serves as its address (more formally, the binary representation of the number serves as the address). In this way, the computer system can refer to the data in "byte #2150" versus the data in "byte #2157," for example. Memory addresses are typically coordinated with the physical layout of the memory system, and so we often portray the numbers in sequential fashion. Figure 5.1 provides such a diagram, with the designated memory address for each byte.

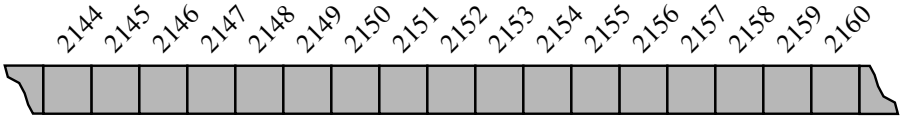


Figure 5.1: A representation of a portion of a computer's memory, with individual bytes labeled with consecutive memory addresses.

Despite the sequential nature of the numbering system, computer hardware is designed, in theory, so that any byte of the main memory can be efficiently accessed based upon its memory address. In this sense, we say that a computer's main memory performs as ***random access memory (RAM)***. That is, it is just as easy to retrieve byte #8675309 as it is to retrieve byte #309. (In practice, there are complicating factors including the use of caches and external memory; we address some of those issues in Chapter 15.) Using the notation for asymptotic analysis, we say that any individual byte of memory can be stored or retrieved in $O(1)$ time.

In general, a programming language keeps track of the association between an identifier and the memory address in which the associated value is stored. For example, identifier `x` might be associated with one value stored in memory, while `y` is associated with another value stored in memory. A common programming task is to keep track of a sequence of related objects. For example, we may want a video game to keep track of the top ten scores for that game. Rather than use ten different variables for this task, we would prefer to use a single name for the group and use index numbers to refer to the high scores in that group.

A group of related variables can be stored one after another in a contiguous portion of the computer's memory. We will denote such a representation as an ***array***. As a tangible example, a text string is stored as an ordered sequence of individual characters. In Python, each character is represented using the Unicode character set, and on most computing systems, Python internally represents each Unicode character with 16 bits (i.e., 2 bytes). Therefore, a six-character string, such as `'SAMPLE'`, would be stored in 12 consecutive bytes of memory, as diagrammed in Figure 5.2.

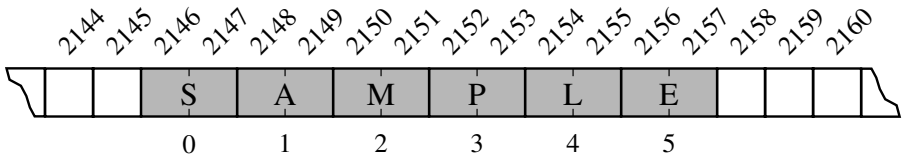


Figure 5.2: A Python string embedded as an array of characters in the computer's memory. We assume that each Unicode character of the string requires two bytes of memory. The numbers below the entries are indices into the string.

We describe this as an *array of six characters*, even though it requires 12 bytes of memory. We will refer to each location within an array as a ***cell***, and will use an integer ***index*** to describe its location within the array, with cells numbered starting with 0, 1, 2, and so on. For example, in Figure 5.2, the cell of the array with index 4 has contents `L` and is stored in bytes 2154 and 2155 of memory.

Each cell of an array must use the same number of bytes. This requirement is what allows an arbitrary cell of the array to be accessed in constant time based on its index. In particular, if one knows the memory address at which an array starts (e.g., 2146 in Figure 5.2), the number of bytes per element (e.g., 2 for a Unicode character), and a desired index within the array, the appropriate memory address can be computed using the calculation, $\mathsf{start} + \mathsf{cellsize} * \mathsf{index}$. By this formula, the cell at index 0 begins precisely at the start of the array, the cell at index 1 begins precisely cellsize bytes beyond the start of the array, and so on. As an example, cell 4 of Figure 5.2 begins at memory location $2146 + 2 \cdot 4 = 2146 + 8 = 2154$.

Of course, the arithmetic for calculating memory addresses within an array can be handled automatically. Therefore, a programmer can envision a more typical high-level abstraction of an array of characters as diagrammed in Figure 5.3.

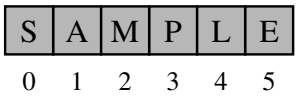


Figure 5.3: A higher-level abstraction for the string portrayed in Figure 5.2.

5.2.1 Referential Arrays

As another motivating example, assume that we want a medical information system to keep track of the patients currently assigned to beds in a certain hospital. If we assume that the hospital has 200 beds, and conveniently that those beds are numbered from 0 to 199, we might consider using an array-based structure to maintain the names of the patients currently assigned to those beds. For example, in Python we might use a list of names, such as:

```
['Rene', 'Joseph', 'Janet', 'Jonas', 'Helen', 'Virginia', ... ]
```

To represent such a list with an array, Python must adhere to the requirement that each cell of the array use the same number of bytes. Yet the elements are strings, and strings naturally have different lengths. Python could attempt to reserve enough space for each cell to hold the *maximum* length string (not just of currently stored strings, but of any string we might ever want to store), but that would be wasteful.

Instead, Python represents a list or tuple instance using an internal storage mechanism of an array of object ***references***. At the lowest level, what is stored is a consecutive sequence of memory addresses at which the elements of the sequence reside. A high-level diagram of such a list is shown in Figure 5.4.

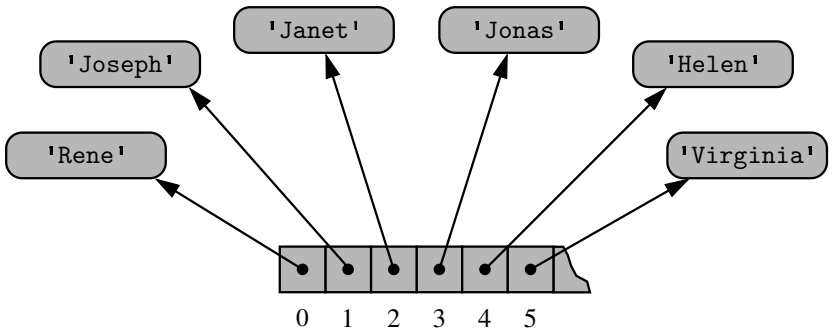


Figure 5.4: An array storing references to strings.

Although the relative size of the individual elements may vary, the number of bits used to store the memory address of each element is fixed (e.g., 64-bits per address). In this way, Python can support constant-time access to a list or tuple element based on its index.

In Figure 5.4, we characterize a list of strings that are the names of the patients in a hospital. It is more likely that a medical information system would manage more comprehensive information on each patient, perhaps represented as an instance of a Patient class. From the perspective of the list implementation, the same principle applies: The list will simply keep a sequence of references to those objects. Note as well that a reference to the **None** object can be used as an element of the list to represent an empty bed in the hospital.

The fact that lists and tuples are referential structures is significant to the semantics of these classes. A single list instance may include multiple references to the same object as elements of the list, and it is possible for a single object to be an element of two or more lists, as those lists simply store references back to that object. As an example, when you compute a slice of a list, the result is a new list instance, but that new list has references to the same elements that are in the original list, as portrayed in Figure 5.5.

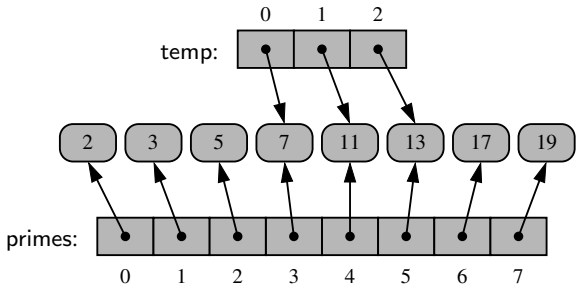


Figure 5.5: The result of the command temp = primes[3:6].

When the elements of the list are immutable objects, as with the integer instances in Figure 5.5, the fact that the two lists share elements is not that significant, as neither of the lists can cause a change to the shared object. If, for example, the command temp[2] = 15 were executed from this configuration, that does not change the existing integer object; it changes the reference in cell 2 of the temp list to reference a different object. The resulting configuration is shown in Figure 5.6.

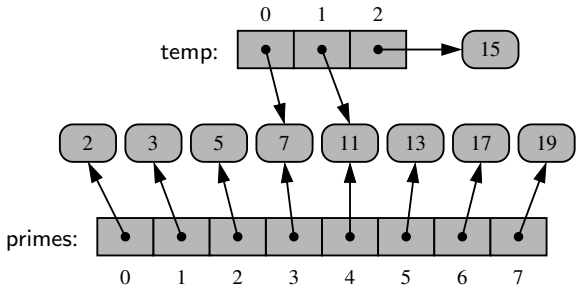


Figure 5.6: The result of the command temp[2] = 15 upon the configuration portrayed in Figure 5.5.

The same semantics is demonstrated when making a new list as a copy of an existing one, with a syntax such as backup = **list**(primes). This produces a new list that is a ***shallow copy*** (see Section 2.6), in that it references the same elements as in the first list. With immutable elements, this point is moot. If the contents of the list were of a mutable type, a ***deep copy***, meaning a new list with *new* elements, can be produced by using the deepcopy function from the copy module.

As a more striking example, it is a common practice in Python to initialize an array of integers using a syntax such as counters $=$ [0] $*$ 8. This syntax produces a list of length eight, with all eight elements being the value zero. Technically, all eight cells of the list reference the *same* object, as portrayed in Figure 5.7.

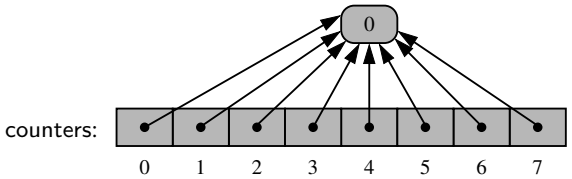


Figure 5.7: The result of the command data $=$ [0] $*$ 8.

At first glance, the extreme level of aliasing in this configuration may seem alarming. However, we rely on the fact that the referenced integer is immutable. Even a command such as counters[2] $+=$ 1 does not technically change the value of the existing integer instance. This computes a new integer, with value $0+1$, and sets cell 2 to reference the newly computed value. The resulting configuration is shown in Figure 5.8.

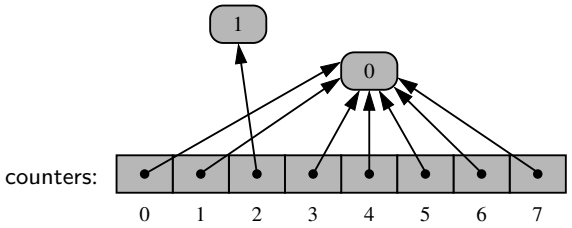


Figure 5.8: The result of command data[2] $+=$ 1 upon the list from Figure 5.7.

As a final manifestation of the referential nature of lists, we note that the extend command is used to add all elements from one list to the end of another list. The extended list does not receive copies of those elements, it receives references to those elements. Figure 5.9 portrays the effect of a call to extend.

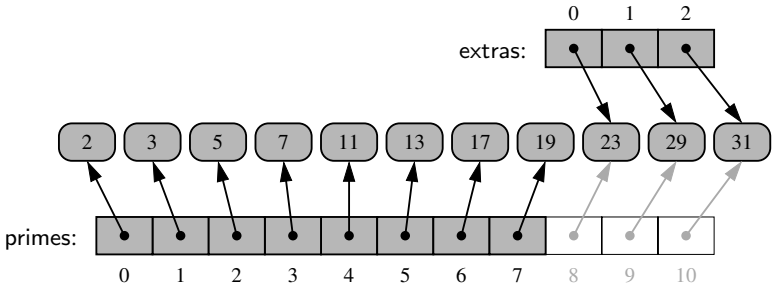
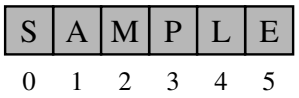


Figure 5.9: The effect of command primes.extend(extras), shown in light gray.

5.2.2 Compact Arrays in Python

In the introduction to this section, we emphasized that strings are represented using an array of characters (not an array of references). We will refer to this more direct representation as a ***compact array*** because the array is storing the bits that represent the primary data (characters, in the case of strings).



Compact arrays have several advantages over referential structures in terms of computing performance. Most significantly, the overall memory usage will be much lower for a compact structure because there is no overhead devoted to the explicit storage of the sequence of memory references (in addition to the primary data). That is, a referential structure will typically use 64-bits for the memory address stored in the array, on top of whatever number of bits are used to represent the object that is considered the element. Also, each Unicode character stored in a compact array within a string typically requires 2 bytes. If each character were stored independently as a one-character string, there would be significantly more bytes used.

As another case study, suppose we wish to store a sequence of one million, 64-bit integers. In theory, we might hope to use only 64 million bits. However, we estimate that a Python list will use *four to five times as much memory*. Each element of the list will result in a 64-bit memory address being stored in the primary array, and an **int** instance being stored elsewhere in memory. Python allows you to query the actual number of bytes being used for the primary storage of any object. This is done using the getsizeof function of the sys module. On our system, the size of a typical int object requires 14 bytes of memory (well beyond the 4 bytes needed for representing the actual 64-bit number). In all, the list will be using 18 bytes per entry, rather than the 4 bytes that a compact list of integers would require.

Another important advantage to a compact structure for high-performance computing is that the primary data are stored consecutively in memory. Note well that this is not the case for a referential structure. That is, even though a list maintains careful ordering of the sequence of memory addresses, where those elements reside in memory is not determined by the list. Because of the workings of the cache and memory hierarchies of computers, it is often advantageous to have data stored in memory near other data that might be used in the same computations.

Despite the apparent inefficiencies of referential structures, we will generally be content with the convenience of Python's lists and tuples in this book. The only place in which we consider alternatives will be in Chapter 15, which focuses on the impact of memory usage on data structures and algorithms. Python provides several means for creating compact arrays of various types.

Primary support for compact arrays is in a module named array. That module defines a class, also named array, providing compact storage for arrays of primitive data types. A portrayal of such an array of integers is shown in Figure 5.10.

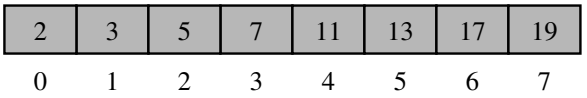


Figure 5.10: Integers stored compactly as elements of a Python array.

The public interface for the array class conforms mostly to that of a Python list. However, the constructor for the array class requires a ***type code*** as a first parameter, which is a character that designates the type of data that will be stored in the array. As a tangible example, the type code, `'i'`, designates an array of (signed) integers, typically represented using at least 16-bits each. We can declare the array shown in Figure 5.10 as,

```
primes = array('i', [2, 3, 5, 7, 11, 13, 17, 19])
```

The type code allows the interpreter to determine precisely how many bits are needed per element of the array. The type codes supported by the array module, as shown in Table 5.1, are formally based upon the native data types used by the C programming language (the language in which the the most widely used distribution of Python is implemented). The precise number of bits for the C data types is system-dependent, but typical ranges are shown in the table.

Code	C Data Type	Typical Number of Bytes
`'b'`	signed char	1
`'B'`	unsigned char	1
`'u'`	Unicode char	2 or 4
`'h'`	signed short int	2
`'H'`	unsigned short int	2
`'i'`	signed int	2 or 4
`'I'`	unsigned int	2 or 4
`'l'`	signed long int	4
`'L'`	unsigned long int	4
`'f'`	float	4
`'d'`	float	8

Table 5.1: Type codes supported by the array module.

The array module does not provide support for making compact arrays of user-defined data types. Compact arrays of such structures can be created with the lower-level support of a module named ctypes. (See Section 5.3.1 for more discussion of the ctypes module.)